西南财经大学天府学院

2026 届

毕业论文（设计）

论文题目：基于边缘计算的藏红花全生长周期智能培育系统设计与实现

学生姓名：杨坤

所在学院：智能科技学院

专　　业：物联网工程

学　　号：42212346

指导教师：姚超

导师职称：讲师

2026 年　3 月

西南财经大学天府学院

毕业论文（设计）答辩修改记录

（装订在封 2）

论文（设计）名　称	基于边缘计算的藏红花全生长周期智能培育系统设计与实现				
学生姓名	杨坤	学 号	42212346	专 业	物联网工程
答辩时间		答辩教室		答辩结论	
1、答辩时，老师提出的修改意见：（学生答辩现场自行记录，答辩后整理记录文字）					
2、修改措施和修改内容总结：（学生答辩后进行论文修改，填写相关内容） （注）：答辩直接通过同学，可以直接记录“无”。 修改后答辩老师签字： 时　　间：					

学术诚信声明

本人郑重声明:所呈交的毕业论文（设计）是本人在导师的指导下取得的成果。除文中已经注明引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写过的作品成果。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。因本毕业论文（设计）引起的法律结果完全由本人承担。

作者签名:

签字日期： 年 月 日

版权使用授权书

本人完全了解西南财经大学天府学院有关保留、使用学位论文(设计)的规定，同意西南财经大学天府学院保留并向国家有关部门或机构送交本毕业论文(设计)的复印件和电子版，允许毕业论文(设计)被查阅和借阅。本人授权西南财经大学天府学院可以将本毕业论文(设计)的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存和汇编本毕业论文(设计)。

作者签名： 导师签名：

签字日期： 年 月 日 签字日期： 年 月 日

基于边缘计算的藏红花全生长周期智能培育系统设计与实现

摘要: 针对传统藏红花培育时环境监测缺少连续性、灌溉依靠人工经验、数据记载和远程操控能力弱等状况来说，创建起一个基于边缘计算的藏红花全生长周期智能培育系统，从而提高培育进程的实时监测、自动控制和智能化程度。系统采用的是设备端、边缘端、云端扩展的总体架构，以 STM32F411 为硬件核心，完成温度、湿度、光照、土壤湿度等环境参数的采集以及执行器的控制、OLED 显示、现场交互；再用树莓派 4B 作为边缘网关，通过 USB 虚拟串口接收设备数据，使用 MicroPython、Flask、SQLite 实现实时监测、历史数据存储、Web 可视化、用户权限管理、自动灌溉策略配置；然后用 MQTT 实现数据的云端扩展和远程访问；最后结合 OpenCV 图像分析和本地轻量模型来提高系统环境评价和辅助决策的能力。从系统测试结果可知，本系统可以较好地完成对藏红花培养环境的实时监测、人工和自动灌溉控制、历史数据可视化、边缘智能分析等工作，具有成本低、部署灵活、扩展性强等特点，可以为特色经济作物小型智能培育提供一种可行的工程实现方案。

关键词： 边缘计算；藏红花；STM32F411；树莓派 4B；自动灌溉

Design and Implementation of an Intelligent Cultivation System for the Entire Growth Cycle of Saffron Based on Edge Computing

Abstract: To address the shortcomings of traditional saffron cultivation, such as lack of continuous environmental monitoring, reliance on manual experience for irrigation, and weak data record and remote control capabilities. an intelligent saffron cultivation system base on edge computing was created to improve realtime monitoring, automatic control; and inteliigence throughout the cultivation process. The system employs an overall architecture of device side, edge side, and cloud based expansion. Using an STM32F411 as the hardware core, It collects environmental parameters such as temperature, humidity, light intensity, and soil moisture, controls actuators, displays data on an OLED, and enables on-site interaction. Raspberry Pi 4B serves as the edge gateway, receiving device data via a USB virtual serial port. MicroPython, Flask, and SQLite are used for realtime monitoring, historical data storage, web visualization, user access management, and automatic irrigation strategy configuration. MQTT is then used for cloudbase data expansion and remote access. Finally, opencv image analysis and local lightweight models are combined to enhance the system's environmental assessment and decisionmaking capabilities. The system test results show that this system can effectively perform task such as realtime monitoring of the saffron cultivation environm ent, manual and automatic irrigation control, historical date visualization, and edge intelligent analysis. It features low cost, flexible deployment, and strong scalability, and can provide a feasible engineering solution for small scale intelligent cultivation of specialty economic crops.

Keywords: Edge computing; Saffron; STM32F411; Raspberry Pi 4B; Automatic irrigation

目 录

1 绪论 1
1.1 研究背景 1
1.2 研究目的及意义 1
1.3 国内外研究现状 2
1.3.1 国内研究现状 2
1.3.2 国外研究现状 3
1.4 研究主要内容 3
2 相关理论与技术基础 4
2.1 物联网与边缘计算相关理论 4
2.2 数据通信与边缘服务相关技术 5
2.2.1 Flask 与 SQLite 边缘服务技术 5
2.2.2 MQTT 云端同步技术 5
2.3 图像分析与轻量边缘智能相关技术 6
2.3.1 基于 OpenCV 的图像颜色分析技术 6
2.3.2 基于本地轻量模型的辅助问答技术 6
3 系统总体方案设计 7
3.1 系统需求及可行性分析 7
3.1.1 需求分析 7
3.1.2 可行性分析 8
3.2 系统的总体方案设计 9
3.3 主要功能模块的器件选型 10
3.4 系统主要元件介绍 11
3.4.1 主控与边缘计算平台 11
3.4.2 环境感知元件 12
3.4.3 人机交互元件 12
3.4.4 执行控制元件 13
3.4.5 图像采集元件 14
4 系统硬件设计 14
4.1 系统整体硬件工作原理 14
4.2 系统功能模块设计 15
4.2.1 传感采集模块设计 15
4.2.2 执行控制模块设计 16
4.2.3 显示与手势交互模块设计 16
4.2.4 边缘网关通信模块设计 17
4.3 系统 PCB 设计 18
5 系统软件设计 19
5.1 系统软件整体设计 19
5.2 设备端程序设计 20
5.3 边缘端服务与数据库设计 21
5.4 图像分析与边缘智能模块设计 23
6 系统测试和结果分析 25

6.1 系统组装与基础测试.....25
6.2 系统功能及性能测试.....26
6.2.1 传感器采集测试.....26
6.2.2 控制执行测试.....28
6.2.3 平台服务与扩展功能测试.....29
6.3 测试结果分析.....33
7 总结和展望.....34
7.1 总结.....34
7.2 展望.....34
7.3 个人总结与反思.....35
参考文献.....36
致谢.....39

1 绪论

1.1 研究背景

随着物联网、嵌入式系统、边缘计算等各方面的迅速发展，农业市场也向着数字化、智能化方向发展。无论是传统的大型农田、小型农业设施，还是小型特色经济作物，都对环境参数的实时性、精度以及运作的灵活性有更高的要求。空气温湿度、光照强度、土壤含水量等都会影响到作物的萌发、生长、开花、品质形成[1]。

藏红花属于有较高经济价值的特色经济作物，它的生命周期存在明显的阶段特点，精细化管理要求比较高。在传统的手工灌溉及管理方式中存在着许多问题，劳动强度大，响应慢，监测次数少，过程不能溯源，环境一致性无法控制。当产业规模扩大或者环境变得更为复杂的时候，人工管理所造成的成本和不稳定性就会急剧增加。因此，建立一个环境感知、边缘处理、自动控制、数据可视化、云端扩展的系统，有较高的研究价值和应用意义。

1.2 研究目的及意义

本文以藏红花全生长周期智能培育为应用背景，设计并实现一套软硬件协同的边缘计算系统，主要目的如下所示。

(1) 用 STM32F411、树莓派 4B 做环境数据的稳定采集、上传、本地处理。

(2) 建立适合藏红花培育环境的监测和控制闭环，对温湿度、光照、土壤湿度等重要参数进行连续感知，可以对水泵、补光灯等执行器进行

控制。

(3) 建立本地可视化管理平台，可以对实时数据进行显示，对历史曲线进行回放，对控制日志进行审计，对用户权限进行管理，对灌溉策略进行配置。

(4) 在传统监测控制的基础上，加入图像分析和轻量级边缘智能辅助模块，使系统的人机交互能力得到提升，并且可以更好的进行应用展示。

本文的研究意义主要表现在, 系统针对具体的作物以及实际的培育场景具有很强的落地性。 从技术实现角度出发，采用设备端、边缘端、云端三层结构来完成系统的搭建，证明了边缘计算在小型智能农业系统中应用的可行性。

1.3 国内外研究现状

1.3.1 国内研究现状

国内对于智慧农业、设施农业以及农业物联网的研究也较多。目前大多数的研究都是从温室环境监测、自动灌溉控制、无线传感网络、农业大数据平台以及作物生长状态识别这几个方面展开的[2]。大多数系统可以完成温湿度、光照、土壤湿度等基本信息的采集，再由单片机、ESP系列模块或者嵌入式 Linux 平台来实现数据上报以及设备控制。 从已有的成果来看，国内的研究在环境监测以及基础自动控制方面比较成熟，但是仍然存在着一些不足，第一，部分系统过于重视数据上传，缺少本地自治控制的能力，在网络出现异常的时候可用性会降低；第二，大部分方案只完成了一个功能的验证，缺少对实际场景的多模块集成；第三，系统大多以常见的蔬菜大棚或者通用盆栽为对象，对于高附加值的特色

作物全周期培育研究较少。另外一些项目虽然实现了页面展示以及硬件联动，但是对数据库的设计、权限的控制、历史的跟踪以及边缘智能的拓展等都没有做深入的研究。

1.3.2 国外研究现状

国外对于精准农业、边缘计算以及农业智能感知的研究开始得比较早，更重视依靠数据驱动的决策支持、边缘自治以及多模态融合。有关的研究一般把低功耗传感设备、边缘计算节点、云平台以及智能分析模型结合在一起，从而达到更加高效地对农业环境进行管理的目的。部分系统在果蔬分级、病虫害识别、灌溉优化、远程运维等各方面取得较好的效果。 在国外的研究中，使用较多的是边缘网关、本地推理、轻量级消息协议、计算机视觉、可视化决策支持平台等。一些研究认为系统具有良好的扩展性、可维护性和离线运行特性，这和边缘计算的思想不谋而合。国外的研究已经由原来的单个传感器的监测发展到现在的“感知-分析-决策-控制”闭环系统的设计。 但是对小型、低成本、适合教学、轻量部署的特色作物培育系统，还需要更加贴近实际应用的工程化方案。因此本文根据藏红花培育的实际需要提出一种结合嵌入式采集、边缘服务、本地数据库、可视化管理以及智能扩展的综合实现方案。

1.4 研究主要内容

本文围绕“基于边缘计算的藏红花全生长周期智能培育系统设计与实现”展开研究，主要内容如下：

(1) 分析藏红花智能培育对环境监测、自动控制、数据管理和远程访问的需求，提出系统总体目标与分层架构。

(2) 完成系统硬件方案设计，确定 STM32F411、树莓派 4B、DHT11、BH1750、电容式土壤湿度传感器、继电器模块、OLED 显示屏、PAJ7620 手势识别模块等器件的选型与连接方案[3]。

(3) 实现设备端程序设计，完成多传感器采集、手势交互、OLED 显示、执行器控制以及 JSON 数据输出。

(4) 实现边缘端软件设计，完成串口数据接收、Web 服务、数据库管理、自动灌溉策略、用户权限管理、历史数据导出和控制日志记录等功能。

(5) 基于摄像头、OpenCV 和轻量级本地语言模型扩展系统智能化能力，提升系统展示性和辅助分析能力。

(6) 对系统进行组装、调试和测试，从功能完整性、运行稳定性、控制效果和扩展能力等角度进行分析总结。

2 相关理论与技术基础

2.1 物联网与边缘计算相关理论

物联网系统一般是由感知层、网络层、应用层三部分构成的。感知层主要是对环境信息进行采集、执行控制，网络层主要是实现设备之间的通信、数据传输，应用层主要是实现数据的管理、业务逻辑的实现。对于智能农业场景来说，单纯的感知和传输不能满足低时延、高可用性要求，所以要采用边缘计算的思想，把一部分存储、分析、控制逻辑放到离设备近的地方。

边缘计算的思想就是把计算、数据处理从中心云端搬到了边缘节点。

边缘计算比完全依靠云平台的方案要响应快、带宽占用小、离线自治能力强、隐私保护更好。本文系统中树莓派是边缘网关，负责数据的接收、数据库的存储、策略的判断、Web 服务以及部分智能分析工作，而云端部分则是作为扩展能力存在的[19]，后面会用到远程同步以及多端访问的功能[4]。该设计既满足本地闭环运行的要求，又为以后系统的扩展留出了余地。

2.2 数据通信与边缘服务相关技术

2.2.1 Flask 与 SQLite 边缘服务技术

Flask 是一种轻量级 Python Web 框架，具有结构清晰、扩展方便和学习成本低等特点，适合中小型边缘服务开发。本文在树莓派上基于 Flask 搭建边缘管理平台，向外提供实时数据查询、历史数据查询、控制下发、策略配置、用户认证和日志查询等接口，并通过 HTML 页面实现可视化展示[5]。

SQLite 作为嵌入式关系数据库，具有零配置、轻量化和易部署的特点，适合单机边缘设备使用[6]。本文使用 SQLite 存储传感历史数据、控制日志、灌溉策略、用户信息、角色信息和设备信息，使系统具备较完整的数据持久化能力。

2.2.2 MQTT 云端同步技术

MQTT 是一种面向低带宽和高延迟网络环境设计的轻量级发布/订阅消息协议，在物联网场景中应用广泛[7]。本文系统在边缘端引入 MQTT 同步机制，当边缘节点连接云端服务器成功后，将实时数据发布到指定主题，通过 MariaDB 持久化在云端完成备份，并能通过扫描二维码或者

访问公网域名来在非局域网查看采集器数据，实现边云协同的数据扩展能力[15]；当云端连接异常时，系统仍可继续在本地完成数据存储与自动控制，从而体现边缘自治优势。

2.3 图像分析与轻量边缘智能相关技术

2.3.1 基于 OpenCV 的图像颜色分析技术

OpenCV 是常用的一种开源计算机视觉库，可以实现图像读取、颜色空间转换、阈值分割、轮廓检测、目标标注等操作。本文使用摄像头获取藏红花现场图像，把图像转换成 HSV 颜色空间，通过颜色区间分割得到红色、绿色、粉色等特征区域的统计结果，从而对花朵的状态进行简单的识别。此方法简单易行，适合用在系统中作为轻量级的视觉分析方案[8]。

2.3.2 基于本地轻量模型的辅助问答技术

在边缘端加上本地语言模型辅助模块，使系统更具有交互性。系统以目前的温度、湿度、光照、土壤湿度为输入参数，产生提示词并调用本地模型得到简短的环境评价建议[16]。该功能本身并不直接参与到控制闭环中来，但是从系统展示性、交互性和智能扩展的角度来说，有积极意义，并且也体现了边缘智能与农业应用相结合的探索方向，本设计使用 qwen2.5 0.5B Q4 量化，非常适合在树莓派这样的轻量级 MPU 上运行[9]。

3 系统总体方案设计

3.1 系统需求及可行性分析

3.1.1 需求分析

为了达到需求指标，紧密结合藏红花实际培育过程中存在的痛点，本系统在设计之初就提出了如下几个方面的核心需求

(1) 环境感知的第一要务就是不断地获得培育室内空气温湿度、光强和土壤含水量等主要参数。这些监测数据不能只在硬件端的屏幕上直接展示出来，还要能够实现用户通过网络远程获取、查看的目的。

(2) 除了允许用户手动去开关水泵、补光灯和指示灯之外，更重要的是应该可以实现自动灌溉。即系统检测到土壤湿度低于设定阈值的时候就会自动开启水泵浇灌。

(3) 在数据存储和管理上，采集到的信息不能过时。系统要有持久化能力，把长期的传感历史记录、设备每次开关的日志、设置好的灌溉策略等都存入数据库中，并可以以文件的形式导出备份。

(4) 在平台交互体验上，需要搭建一个可视化的 Web 服务端。管理员登录到该网页之后，不但可以查看到实时刷新的动态监控大屏以及历史折线图，还可以对后台参数进行修改，并且可以对不同的用户进行权限划分。

(5) 把边缘计算概念落到实处、打通云端同步链路、巧妙地将 OpenCV 图像特征提取和本地大模型问答功能融合在一起。

(6) 从工程落地、实现角度出发，为了保证实物制作、答辩演示顺利进行，硬件接线要尽量简洁可靠，不能出现接触不良的情况，软件代码

也要保证清晰的分层结构。整体设计出来的系统要好测试、好演示，并且便于日后加装新的传感器或者模块。

3.1.2 可行性分析

技术可行。从硬件方案来说，不管是作为主控的 STM32F411 和树莓派 4B，还是外围的 DHT11、BH1750、电容土壤湿度计、OLED 等组件，都是市场上十分成熟并且开源资料十分丰富的通用模块。在软件架构上，下位机使用了 MicroPython 编程[10]，大大缩短了底层硬件的调试时间；上位机边缘端 直接依靠 Python 生态，在 Flask 框架的基础上搭建起了后端，使用 SQLite 进行数据处理，利用 OpenCV 完成视觉任务。该套软硬结合的技术栈已经得到了很多行业的落地项目验证，可以保证本系统的各种复杂的业务能顺利进行。

经济可行性。本系统所用到的主要零部件绝大部分都是基础电子教学、实验模块。这些配件价格便宜。另外，直接使用树莓派作为算力中心，并且配备现成的传感器成品，不但可以节省从零开始绘制复杂的 PCB 和贴片打样所付出的巨大成本，而且可以避免初期投入过多。

操作可行性。用户日常使用时只需用浏览器打开 Web 后台就可以完成所有环境监控、设备控制，界面逻辑非常直观。树莓派和 STM32 之间使用 USB 线以串行方式进行通信，并且使用免安装的本地数据库，基本上实现了插电即用的效果。系统自带的自动灌溉方案、完善的日志存档系统等都会大大减少对人工操作的需求，该系统非常容易上手并且视觉反馈清晰。

3.2 系统的总体方案设计

本文采用“设备端、边缘端、云端扩展”三层结构来设计系统的总体方案。设备端采用 STM32F411 和各种传感器、执行器，完成数据采集、现场显示、手势交互、执行控制[20]；边缘端用树莓派 4B 接收设备数据、提供 Web 服务、存储历史信息、执行自动灌溉策略、调用图像分析和本地问答模块；云端部分使用 MQTT 进行数据同步和远程扩展。

设备端每隔一段时间就会采集环境数据并以 USB 虚拟串口的形式发送 JSON 数据到边缘端；边缘端串口线程实时监听并解析数据，将数据写入 SQLite 数据库，同时更新内存中最新的状态；前端页面通过 API 获取最新的数据以及历史数据，从而达到实时监控和图表展示的目的；当检测到土壤湿度低于阈值的时候，边缘端会自动开启水泵，并且在达到设定的时间之后再自动关闭水泵；当云端连接正常的时候，边缘端会把数据通过 MQTT 发布到云端，从而实现边云协同。

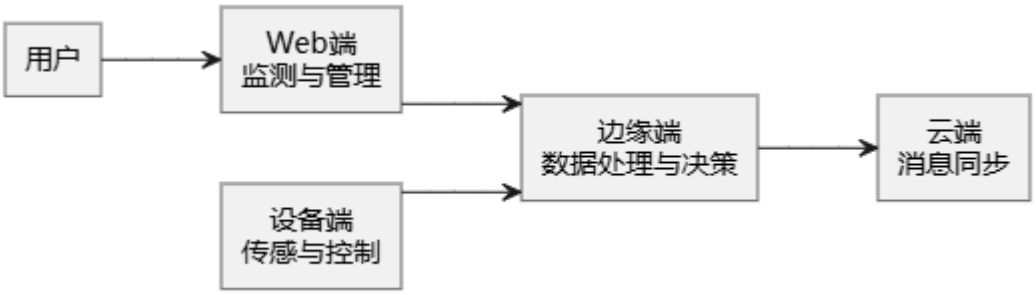

图 3.1 系统总体架构图

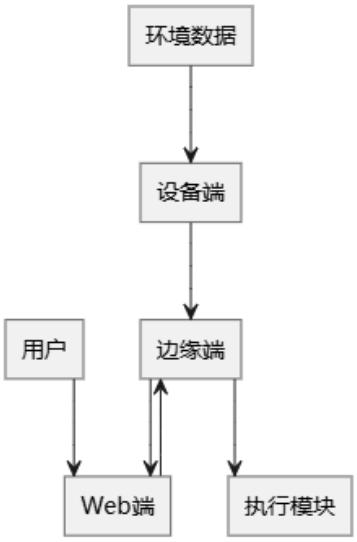

图 3.2 系统数据流与控制流示意图

3.3 主要功能模块的器件选型

系统器件选型不仅影响功能实现，还直接关系到调试难度、成本控制和后续扩展性。结合项目实际条件，本文最终确定的主要器件如表 3.1 所示。

表 3.1 主要器件选型表

模块	选用器件	主要作用	选型理由
主控单元	STM32F411CEU6	传感器数据采集、执行控制、现场交互	性能较强，支持 MicroPython，便于扩展
边缘网关	Raspberry Pi 4B 4GB	Web 服务、数据库、策略执行、图像分析	Linux 环境成熟，原生 Python 适合机器学习
温湿度采集	DHT11	采集空气温度与湿度	接口简单，成本低
光照采集	BH1750	采集光照强度	I2C 数字输出，数据读取方便
土壤湿度采集	电容式土壤湿度传感器	采集土壤含水状态	抗腐蚀能力较好，适合长期放置
显示模块	OLED 显示屏	本地数据显示与状态显示	功耗低，显示清晰，I2C 接口简单
交互模块	PAJ7620 手势传感器	本地翻页与设备切换控制	方便用户在脏手的状态下查看数据
执行模块	继电器 + 水泵 + LED 灯带	灌溉与补光控制	驱动方式成熟，扩展方便
图像采集	树莓派 USB 摄像头	拍照与图像分析	与边缘端易集成，适合展示
数据同步	MQTT	云端扩展通信	协议轻量，适合物联网场景

表 3.2 备选方案对比表

功能	方案一	方案二	最终选择	选择原因
主控芯片	STM32F103	STM32F411	STM32F411	运算资源更充足，扩展性更好
边缘节点	ESP32	树莓派 4B	树莓派 4B	更适合部署 Web, 数据库与 AI 模块
光照传感器	模拟光敏电阻	BH1750	BH1750	数字输出，标定更方便
数据库存储	仅文件存储	SQLite	SQLite	查询方便, 结构化程度更高
云数据库存储	仅文件存储	MariaDB	MariaDB	稳定性高, 易于存储

3.4 系统主要元件介绍

为实现藏红花全生长周期培育过程中的环境监测、智能控制和数据管理功能，系统硬件由主控与边缘计算平台、环境感知元件、人机交互元件、执行控制元件以及图像采集元件等部分组成。各类元件相互配合，形成了从数据采集、信息处理到设备执行的完整链路。

3.4.1 主控与边缘计算平台

本系统用 STM32F411 做底层主控制器，用树莓派 4B 做边缘计算平台。STM32F411 集成了较多的 GPIO、ADC、I2C、USART 等外设，可以完成温湿度、光照、土壤湿度等传感器的数据采集，并且可以实现 OLED 显示、手势识别以及继电器控制等功能[11]。树莓派 4B 主要完成边缘侧的数据接收、存储、可视化显示和控制策略执行等任务，可以安装 Flask、SQLite 和 OpenCV 等软件环境，满足系统本地数据处理及功能扩展的要求。两者一起工作，系统既有单片机的实时控制能力，又有边缘设备的数据处理能力。

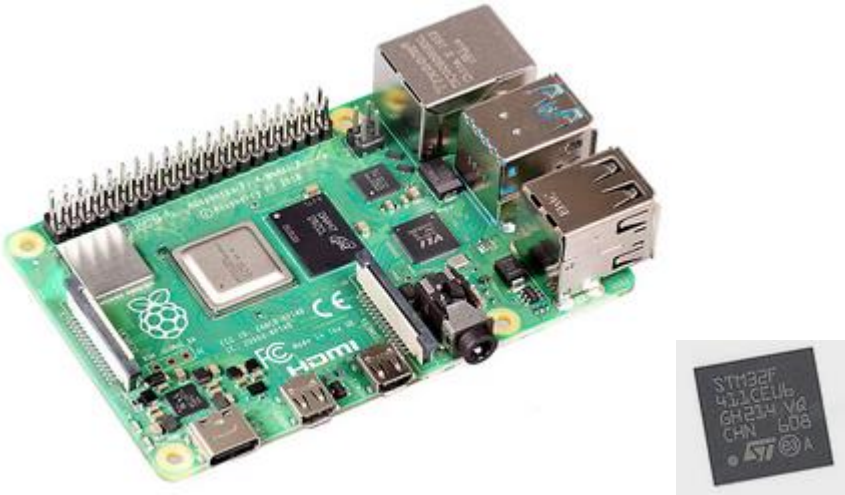

图 3.3 树莓派 与 STM32F411

3.4.2 环境感知元件

系统环境感知部分主要是 DHT11 温湿度传感器、BH1750 光照传感器、电容式土壤湿度传感器。DHT11 用来采集空气温度、湿度信息，可以反映培育环境的基本状况[12]；BH1750 用来检测环境光照强度，它的输出是数字信号，接口简单，适合直接和主控相连；电容式土壤湿度传感器用来检测培育基质的含水情况，把模拟信号输入 STM32 的 ADC 通道进行采样。上述传感器一起形成了系统的环境监测基础，给后面自动灌溉、补光控制提供数据支撑。

图 3.4 温湿度传感器、光照传感器和电容式土壤湿度传感器

3.4.3 人机交互元件

为了提高系统的本地交互能力，本文选择了 OLED 显示模块、

PAJ7620 手势识别模块。OLED 显示模块体积小、功耗低、显示清晰，可以用来实时显示温度、湿度、光照、土壤湿度和系统状态等信息。PAJ7620 手势识别模块可以采用 I2C 接口和主控相连，从而实现非接触式的操作，方便用户进行页面的切换或者功能的选择[13]。两者结合之后，可以提高系统的运行状态可视化程度以及交互便捷性。

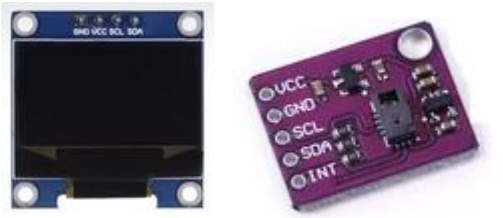

图 3.5 OLED 显示屏与手势识别模块

3.4.4 执行控制元件

系统的执行控制部分主要是由继电器模块、水泵、LED 灯带组成。继电器模块起到单片机和外部负载之间隔离的作用，使主控可以安全地控制水泵和补光设备。水泵是系统自动灌溉功能的执行机构，当土壤湿度小于设定值的时候，系统会自动开启水泵进行补水。LED 灯带主要用作环境补光、状态指示，光照不足或者功能演示时起作用。加入执行控制元件之后，系统就具备了按照环境数据自行调节培育条件的功能。

图 3.6 水泵与灯带

3.4.5 图像采集元件

为了提高系统对藏红花生长状况的感知能力，本文使用了 USB 摄像头做为图像采集器件。该模块可以直接和树莓派相连，用来采集作物生长图像，再用 OpenCV 完成图像预处理以及特征分析。相比于传统的环境参数监测来说，摄像头的加入使得系统有了视觉信息的获取能力，为后面进行花朵状态识别、生长过程分析打下了基础。

因此，本文所选用的各个元件在功能上分工明确，可以共同支撑藏红花智能培育系统。主控和边缘平台完成数据处理、系统调度，传感器完成环境信息采集，人机交互元件完成状态显示和操作输入，执行元件完成自动调节，图像采集元件给系统智能化扩展提供支持。

4 系统硬件设计

4.1 系统整体硬件工作原理

系统硬件由传感采集单元、执行控制单元、显示交互单元、边缘通信单元和供电单元构成。STM32F411 作为底层主控，负责读取 DHT11 温湿度数据、BH1750 光照数据和电容式土壤湿度模拟量，并将处理后的结果显示在 OLED 上，同时以 JSON 格式经 USB 虚拟串口发送给树莓派。树莓派接收数据后进行存储与服务处理，并根据前端操作或自

动灌溉策略向 STM32 回发控制命令。

执行控制部分主要是水泵继电器和 LED 灯带继电器。当边缘端发出控制命令之后，STM32 解析出对应的 JSON 指令，并且控制继电器的状态，从而达到灌溉、补光的效果。手势识别模块 PAJ7620 和 OLED 共同构成本地人机交互单元，用户可以对设备进行直接的页面切换以及控制。系统总体硬件连接关系清楚，适合做原型设计和调试。

根据当前程序设计，主要接口关系如下：

(1) DHT11 接入 STM32 的 A1 引脚；

(2) 电容式土壤湿度传感器模拟量接入 A2 引脚；

(3) I2C 总线用于连接 BH1750、OLED 和 PAJ7620，其中 SCL 接 PB6，SDA 接 PB7；

(4) 水泵继电器控制引脚接 B10；

(5) LED 灯带继电器控制引脚接 B12；

(6) 板载状态指示灯使用 C13；

(7) STM32 与树莓派通过 USB 连接，实现供电与串口通信。

4.2 系统功能模块设计

4.2.1 传感采集模块设计

传感采集模块由 DHT11 温湿度传感器、BH1750 光照传感器、电容式土壤湿度传感器组成。DHT11 用单总线数字通信方式输出温湿度信息，BH1750 用 I2C 接口和主控连接，可以完成光照强度的数字化采集，土壤湿度传感器输出模拟电压信号，经过 STM32ADC 模块采样转换成百分比的形式。

为了使系统更加适应，在软件中加入了数据有效性和标定转换的逻辑。土壤湿度数据不是直接用 ADC 值，而是用干燥值和湿润值的标定区间做线性映射得到百分比结果。设计之后，使得系统更符合实际培育场景使用习惯。

4.2.2 执行控制模块设计

执行控制模块主要是水泵、继电器驱动模块、LED 灯带。水泵用来在土壤湿度过低的时候自动补水，LED 灯带用来补光或者展示控制，继电器模块作为控制接口，可以对外部负载进行安全驱动。STM32 根据接收到的命令控制继电器引脚的高低电平来控制负载的启停。

边缘端线程不断读取当前土壤湿度和灌溉策略参数，当满足触发条件的时候，就会向设备端发出开启水泵的指令，在达到设定的灌溉时间之后，再发出关闭指令。系统还要对控制日志以及最近的动作时间加以记载，从而给状态追踪以及测试分析赋予支撑。

4.2.3 显示与手势交互模块设计

为了提高现场的可视化效果，在系统中加入了 OLED 显示模块来显示实时温度、湿度、光照、土壤湿度和手势信息。显示页面采用分页结构，主监测页、控制页、信息页等都有对应的页面，用户用手势进行页面的跳转。

PAJ7620 手势识别模块用 I2C 和 STM32 相连，可以识别出向左、向右、向前、向后、向上、向下等动作。系统把手势识别的结果同菜单逻辑结合起来，实现了不用按键就能翻页、选中并执行控制的功能，大大提高了系统的交互性, 让用户在易脏手的培育室环境也能查看数据。

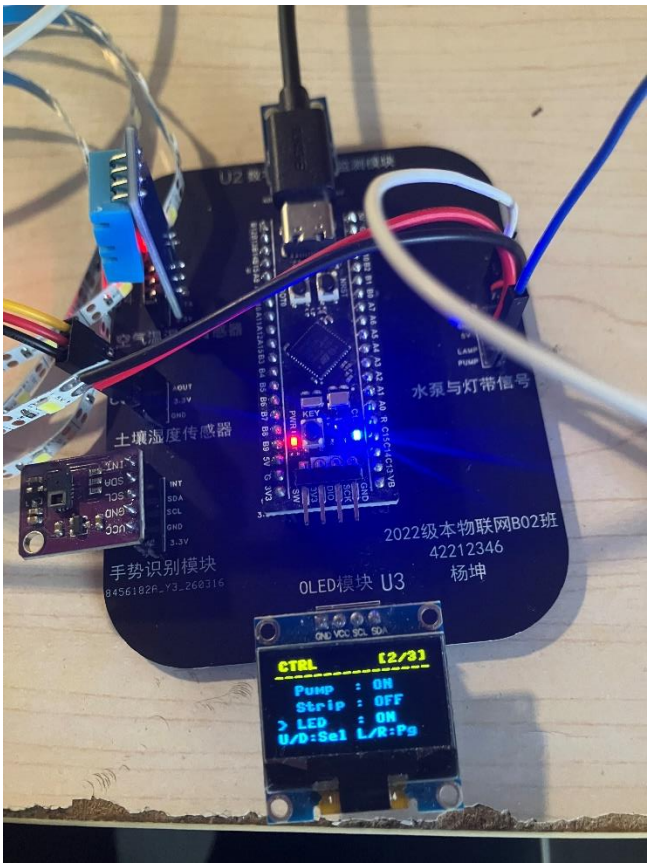

图 4.1 主要模块展示

4.2.4 边缘网关通信模块设计

边缘网关通信模块由 STM32 和树莓派之间 USB 虚拟串口组成。该种方式可以同时解决供电和通信的问题，省去了另外加装无线传输模块的调试复杂程度。系统运行的时候，STM32 周期性地发出 JSON 数据帧，树莓派用串口线程一直监听数据并解析数据，从而达到环境状态的同步。

相比于蓝牙或者 Wi-Fi 直连的方式，在原型系统中使用 USB 串口链路更稳定、调试更简单，适合原型设计阶段快速落地。同时边缘端仍然具有 MQTT 云同步的功能，为之后的网络化扩展做好准备。

4.3 系统 PCB 设计

为进一步梳理系统硬件连接关系，在完成面包板功能验证后，本文基于 STM32F411 最小系统绘制了系统核心接口原理图，并进行了 PCB 版图规划。设计中对光照、空气温湿度、土壤湿度、手势识别、OLED 显示以及水泵和灯带控制等接口进行了统一整理，为后续硬件集成提供参考。

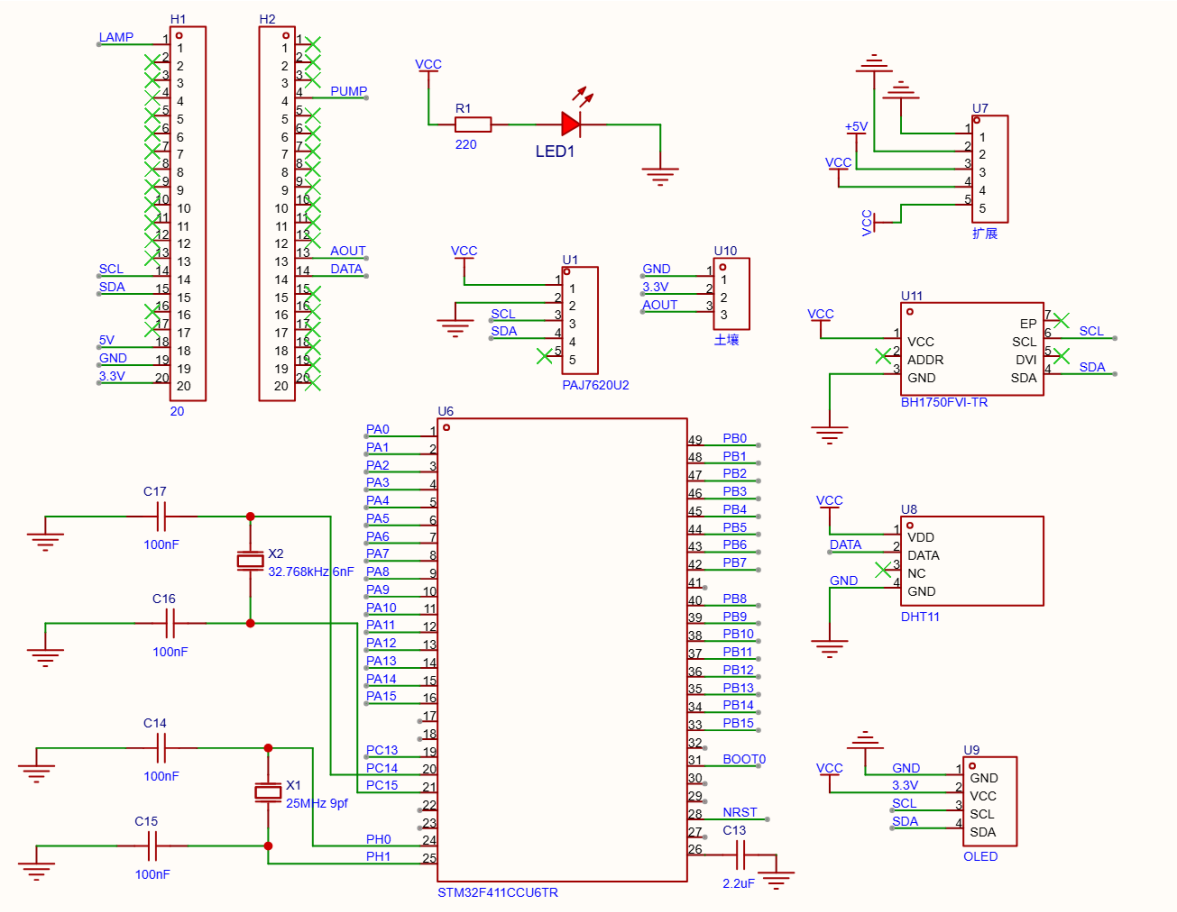

图 4.2　PCB 原理图

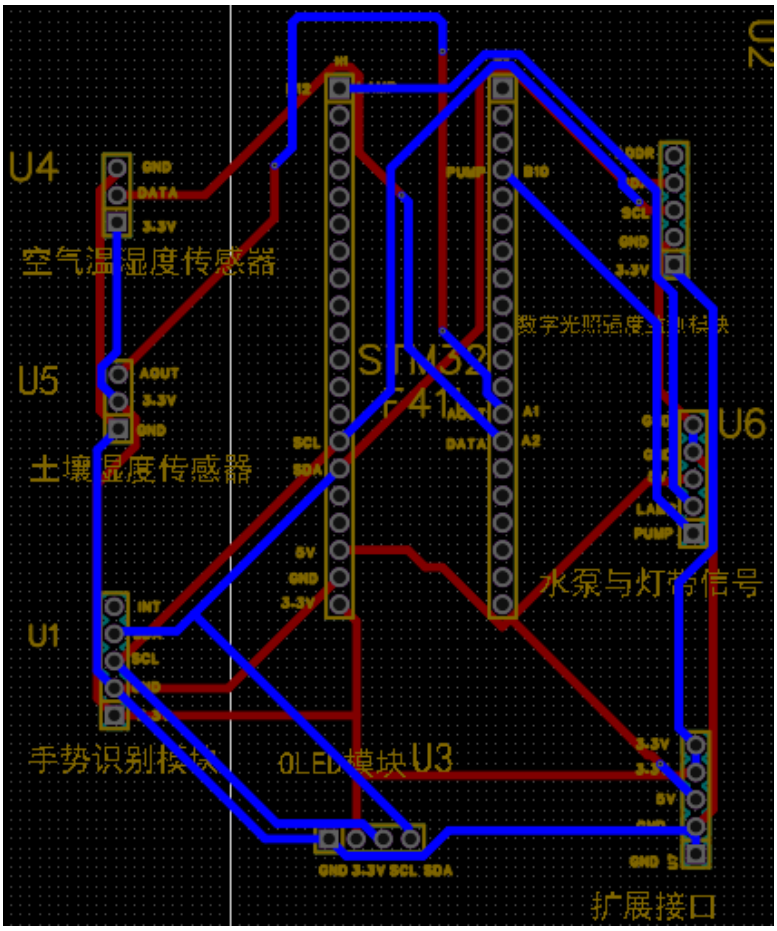

图 4.3　PCB 板图

5 系统软件设计

5.1 系统软件整体设计

系统软件分为设备端软件和边缘端软件。设备端的软件运行在 STM32F411 的 MicroPython 环境当中，对多个传感器实施采集工作，并且更新 OLED 屏幕内容、识别手势动作以及操控执行器装置，最后把串口数据发送出去。边缘端软件运行在树莓派 4B 的 Python 环境中，使用 Flask 框架实现 API 和服务页面的开发，使用 SQLite 进行数据持久化，用后台线程来完成串口监听和自动灌溉策略的执行[14]。

5.2 设备端程序设计

设备端程序采用 MicroPython 实现，整体结构包括硬件初始化、页面显示、命令处理和主循环四个部分。系统启动后，首先初始化 DHT11、BH1750、OLED、PAJ7620、土壤湿度 ADC、水泵继电器和灯带继电器等外设；初始化完成后进入主循环，以非阻塞方式完成手势处理、串口命令处理和周期性传感器采样。

程序用 select.poll 机制轮询串口输入，防止等待命令的时候阻塞其他任务，主循环中设置 1s 的采样周期，定时读取传感器数据并组装成 json 数据包，手势识别模块用冷却时间和显示超时来减少重复触发。该设计可以保证设备端在资源有限的情况下，仍然可以比较平稳地完成多任务协同。 设备端主要功能流程可概括如下：

(1). 初始化各类外设并显示系统启动状态；

(2). 轮询手势识别结果，处理页面切换与本地控制；

(3). 非阻塞监听串口命令，解析控制 JSON 并切换执行器状态；

(4). 周期性采集温度、湿度、光照和土壤湿度数据；

(5). 将采集结果显示于 OLED，并通过串口输出至树莓派。

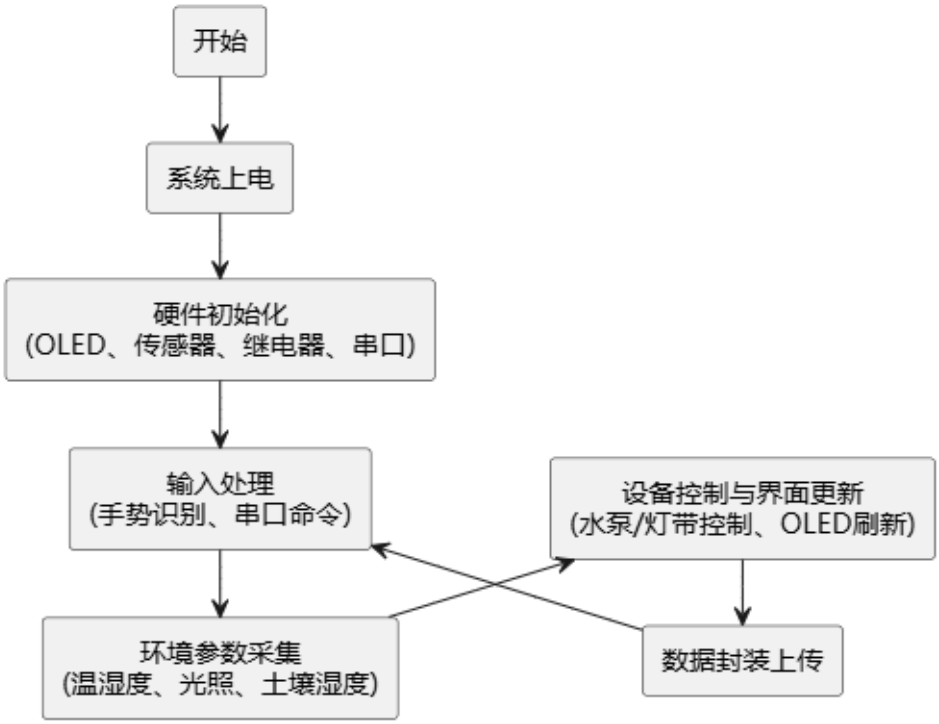

图 5.1 STM32 设备端主程序流程图

5.3 边缘端服务与数据库设计

边缘端服务用 Flask 搭建起来，包含串口数据接收线程、自动灌溉线程、RESTful API、Web 页面渲染、认证授权、数据库管理、云同步扩展等诸多模块[17]。串口线程对 STM32 发来的 JSON 数据进行解析，把最新的数据存入内存和数据库中，自动灌溉线程根据设定的阈值和冷却时间来决定是否启动水泵，前端页面通过 API 获取实时数据、历史数据和控制日志，管理员可以对灌溉策略进行配置[18]。

根据目前项目实现，系统给出较为完备的接口集，包含实时数据查询、历史数据查询、历史数据导出、控制下发、策略查询及保存、控制日志查询、用户注册登录、当前用户信息查询、设备状态查询、拍照和图像分析、边缘问答等。其主要接口可以整理成表 5.1。

表 5.1 主要接口设计表

接口路径	请求方式	主要功能
/api/v1/sensors/latest	GET	获取最新环境数据
/api/v1/sensors/history	GET	获取历史传感数据
/api/v1/sensors/history.csv	GET	导出历史数据 CSV
/api/v1/control	POST	下发设备控制命令
/api/v1/control/logs	GET	查询控制日志
/api/v1/policy/irrigation	GET/POST	查询或设置灌溉策略
/api/v1/policy/irrigation/status	GET	查询自动灌溉状态
/api/v1/auth/me	GET	查询当前用户信息
/api/v1/devices/status	GET	查询设备状态
/api/v1/vision/analyze	POST	图像分析
/api/v1/assistant	POST	边缘智能问答

本文在边缘端用 SQLite 建立了很多业务表。数据库除了保存环境数据之外，还会记录用户、角色、设备、控制日志以及灌溉策略等信息。其主要的表结构如表 5.2 所示。

表 5.2 数据库主要表设计

数据表名	主要字段	作用说明
users	id、username、password_hash、created_at	存储用户账户信息
roles	id、name	存储角色定义
user_roles	user_id、role_id	实现用户与角色关联
devices	id、name、description、last_seen	存储设备基本信息
sensor_data	id、device_id、temperature、humidity、lux、soil、timestamp	存储历史环境数据
control_logs	id、device_id、actuator、action、raw_command、success、created_at	存储控制记录
irrigation_policies	id、device_id、enabled、soil_threshold_min、watering_seconds、cooldown_seconds、updated_at	存储灌溉策略参数

此外，系统在数据库层设计了相应触发器，当控制日志插入后，能够自动更新设备的最近在线时间。该设计体现了数据库对业务逻辑的支撑作用，保证了设备状态信息更新的及时性和一致性，提高了系统的数据管理能力。

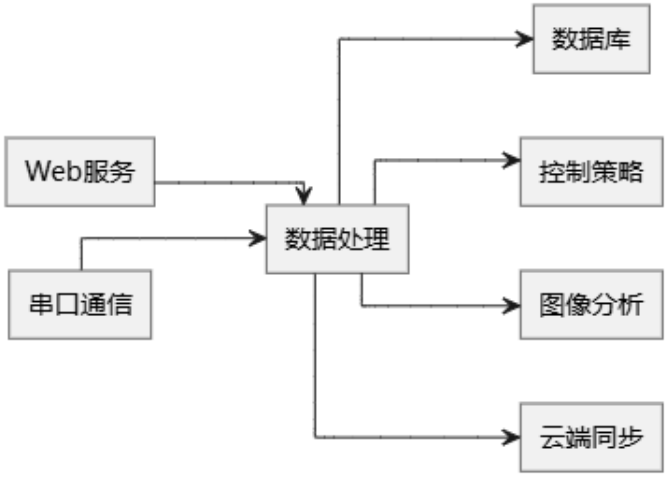

图 5.2 边缘端软件模块图

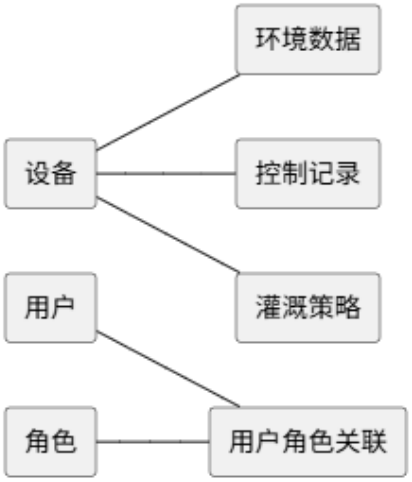

图 5.3 数据库 E-R 图

5.4 图像分析与边缘智能模块设计

在基础监测控制功能的基础上又加上了图像采集、图像分析、本地问答这三个扩展模块。拍照模块调用树莓派摄像头接口来完成图片抓拍；图像分析模块使用 OpenCV 把图像转换成 HSV 颜色空间，按照颜色区间统计和轮廓框选来识别花朵的颜色特征，把识别结果映射到花蕾期、盛开期或者成熟、凋谢期这些状态上。

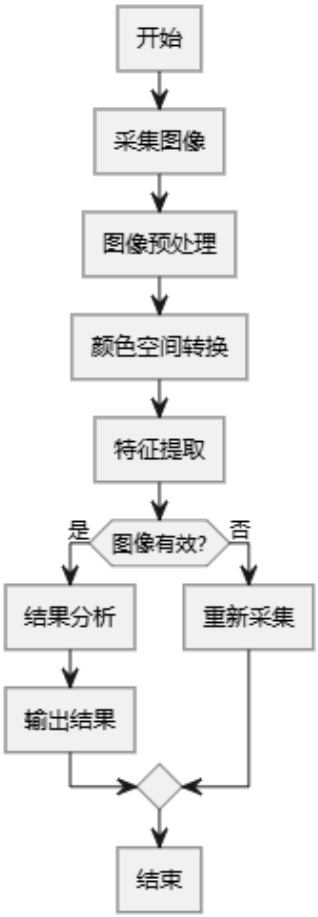

图 5.4 图像分析流程图

```
def _refine_mask(self, mask):
    short_side = min(mask.shape[:2])
    open_size = self._ensure_odd(min(5, max(3, short_side // 180)))
    close_size = self._ensure_odd(min(17, max(7, short_side // 45)))
    bridge_size = self._ensure_odd(min(31, max(11, short_side // 28)))

    open_kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (open_size, open_size))
    close_kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (close_size, close_size))
    bridge_kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (bridge_size, bridge_size))

    cleaned = cv2.morphologyEx(mask, cv2.MORPH_OPEN, open_kernel)
    cleaned = cv2.morphologyEx(cleaned, cv2.MORPH_CLOSE, close_kernel)
    return cv2.morphologyEx(cleaned, cv2.MORPH_CLOSE, bridge_kernel)
```

图 5.5 核心图像预处理代码

边缘智能问答模块会根据目前的环境数据生成提示词，然后调用本地轻量级模型给出简短的环境评价意见。

```
(.venv) rpi@rpi:~/fyp $ python test_llm.py

Using chat eos_token: <|im_end|>
Using chat bos_token: <|endoftext|>

模型加载成功！C++ 引擎没问题！
正在测试推理...
llama_perf_context_print:        load time =     251.86 ms
llama_perf_context_print: prompt eval time =     251.37 ms /     4 tokens (   62.84 ms per
 token,    15.91 tokens per second)
llama_perf_context_print:        eval time =    1140.86 ms /     9 runs   (  126.76 ms per
 token,     7.89 tokens per second)
llama_perf_context_print:       total time =    1408.71 ms /    13 tokens
llama_perf_context_print:    graphs reused =          8
生成结果：您是否是人工智能。如果是，那我需要
(.venv) rpi@rpi:~/fyp $
```

图 5.6　llm 推理模块

以上模块加入之后，系统就具备了实用的功能。尽管目前的图像分析方法还属于规则型轻量级识别，模型推理能力也主要起到辅助展示的作用，但是整体方案已经具有了传感监测、图像感知、语言辅助的多模态边缘智能特征，为系统后期升级打下了基础。

图 5.7　Web app 页面

6 系统测试和结果分析

6.1 系统组装与基础测试

系统测试前先做硬件连接检查、供电检查、串口识别检查、软件部

署检查。设备端通过 USB 与树莓派连接之后，应该能够被边缘端识别出相应的串口设备；系统启动之后，OLED 应该可以正常显示实时数据页面；树莓派 Web 服务启动之后，可以通过浏览器访问实时监控页面、历史页面以及后台管理页面。

基础测试阶段主要是对传感器能否正常返回数据、继电器是否可以响应本地手势或者远程控制、边缘端是否可以把串口数据写入数据库、前端页面是否可以实时刷新、自动灌溉线程是否可以根据策略正常触发进行检查。使用 PCB 方案时还要检查 PCB 上电后各个接口的工作是否正常，焊点是否可靠，外设插接是否稳定。

```
Port /dev/ttyACM0, 08:21:32 [U]

Press CTRL-A Z for help on special keys

{"humi": 49, "timestamp": 363698, "gesture": null, "temp": 20, "lux": 210.0, "cycle": 355}
{"humi": 48, "timestamp": 364724, "gesture": "向右", "temp": 20, "lux": 968.3, "cycle": 356}
{"humi": 48, "timestamp": 365731, "gesture": "向左", "temp": 20, "lux": 144.2, "cycle": 357}
{"humi": 47, "timestamp": 366738, "gesture": "向右", "temp": 20, "lux": 243.3, "cycle": 358}
{"humi": 47, "timestamp": 367752, "gesture": "向右", "temp": 20, "lux": 9.2, "cycle": 359}
{"humi": 49, "timestamp": 368766, "gesture": "向右", "temp": 20, "lux": 196.7, "cycle": 360}
{"humi": 53, "timestamp": 369780, "gesture": null, "temp": 21, "lux": 153.3, "cycle": 361}
```

图 6.1 串口监视与页面运行

6.2 系统功能及性能测试

为了检验系统的有效性，本文对环境采集功能、控制执行功能、自动灌溉功能、历史数据管理功能、边云同步功能和图像分析功能等进行了测试。

6.2.1 传感器采集测试

该测试主要是对 DHT11、BH1750、土壤湿度传感器数据读取的稳定性进行检验，并且对边缘端页面显示和数据库写入情况进行检查。在测试过程中可以运行系统几十分钟到几十个小时，在测试过程中对系统数据进行更新记录，对异常中断次数进行统计记录，对数据库的入库情况

进行记录。

表 6.1　传感器采集测试结果表

测试项目	测试条件	预期结果	实测结果
温度采集	室外常温环境	正常返回温度值	24℃
湿度采集	室外常湿环境	正常返回湿度值	57%
光照采集	室外白天	光照值明显变化	192lux
土壤湿度采集	湿土环境	正常返回土壤湿度值	53%

综合表的采集结果,本系统在室外湿土，常温，常湿的实测物理量采集数据实物图如下:

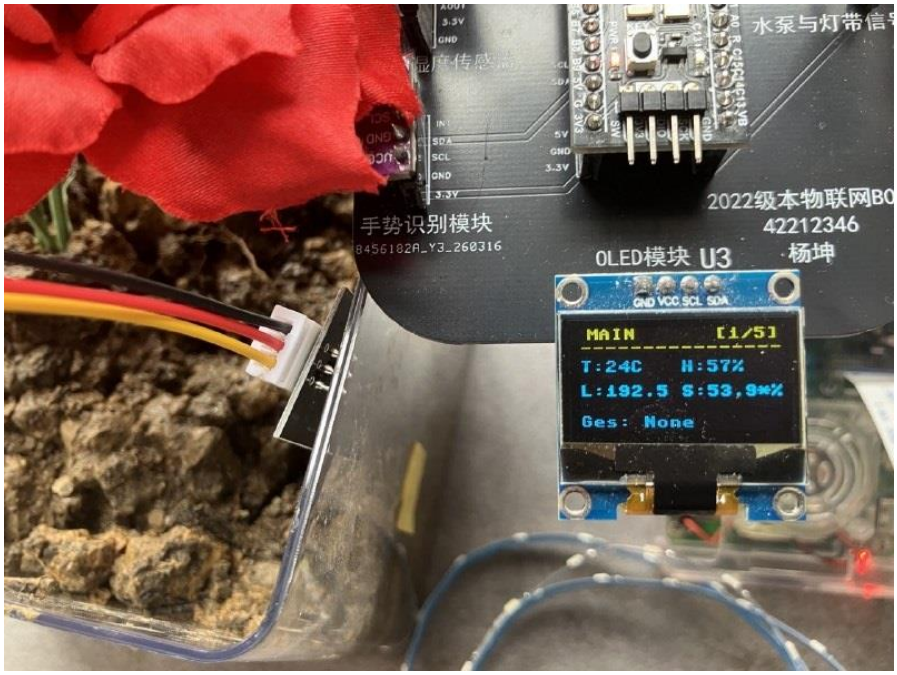

图 6.2　传感器多物理量状态采集实测图

系统连续运行 3 小时 8 分,共向数据库写入 8260 条数据

```
MariaDB [saffron_db]> SELECT MIN(timestamp) AS first_ts,
    ->        MAX(timestamp) AS last_ts,
    ->        COUNT(*) AS total_rows
    -> FROM cloud_sensor_data
    -> WHERE timestamp BETWEEN '2026-03-14 20:30:12' AND '2026-03-14 22:38:43';
+---------------------+---------------------+------------+
| first_ts            | last_ts             | total_rows |
+---------------------+---------------------+------------+
| 2026-03-14 20:30:12 | 2026-03-14 22:38:43 |       8260 |
+---------------------+---------------------+------------+
1 row in set (0.005 sec)
```

图 6.3　数据库实验数据

6.2.2 控制执行测试

本测试主要对水泵、LED 灯带、警告指示灯的自动控制进行检验。测试时可以在后台页面设置土壤湿度阈值、浇水时长、冷却时间等参数，测试平台采用 postman 与观察实物状态。

表 6.2 控制与自动灌溉测试结果表

测试项目	操作方式	预期结果	实测结果
智能灌溉	下发开启自动模式	水泵根据策略触发灌溉	水泵成功启动
LED 灯带控制	页面下发开关命令	灯带状态正确变化	灯带成功开启与关闭
本地手势控制	在控制页执行手势	对应执行器状态变化	OLED 和 WebUI 显示手势方向

为了直观地看到智能灌溉策略的效果，本文给出如下图 6.4 所示的实物测试环境。测试时先在 Web 后台设置规则，再下达开启自动模式的命令。控制策略才会被激活。然后控制指令发送到设备端，继电器闭合驱动水泵开始工作，向培养区域注水，在达到预设的浇水时间之后自动停止，完成整个灌溉闭环。

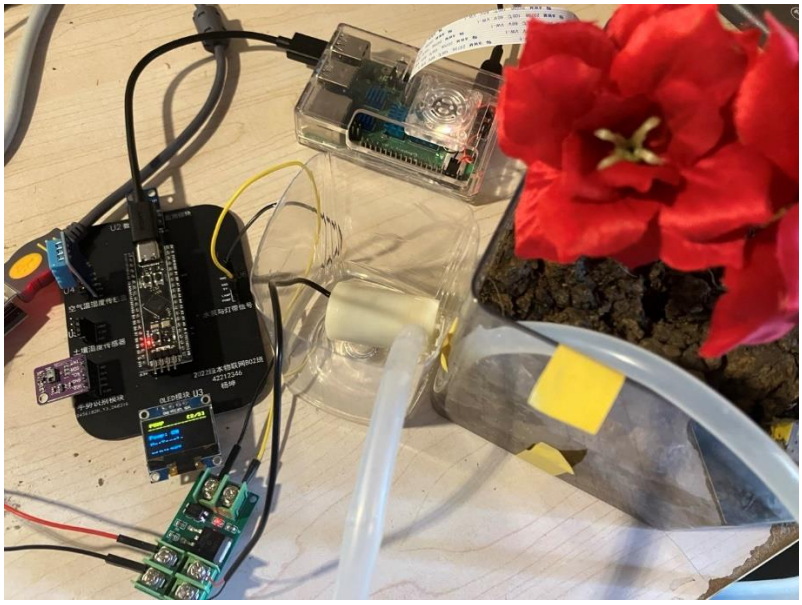

图 6.4 智能灌溉策略下水泵自动触发测试

以下进行的手势识别测试很好地证明了系统的响应是实时的、准确的。用户不用去碰触设备，在界面上的操作只需要手势就能完成。从图 6.5 可知，当用户向右手势的时候，OLED 屏幕成功从主监控界面切换到水

泵控制界面，然后在水泵控制界面执行向上手势，系统正确地接收到指令并自动打开水泵。除此之外，WebUI 后台可以实时同步并且更新目前识别出的手势动作。该结果说明系统的本地交互模块运行正常、反应快。

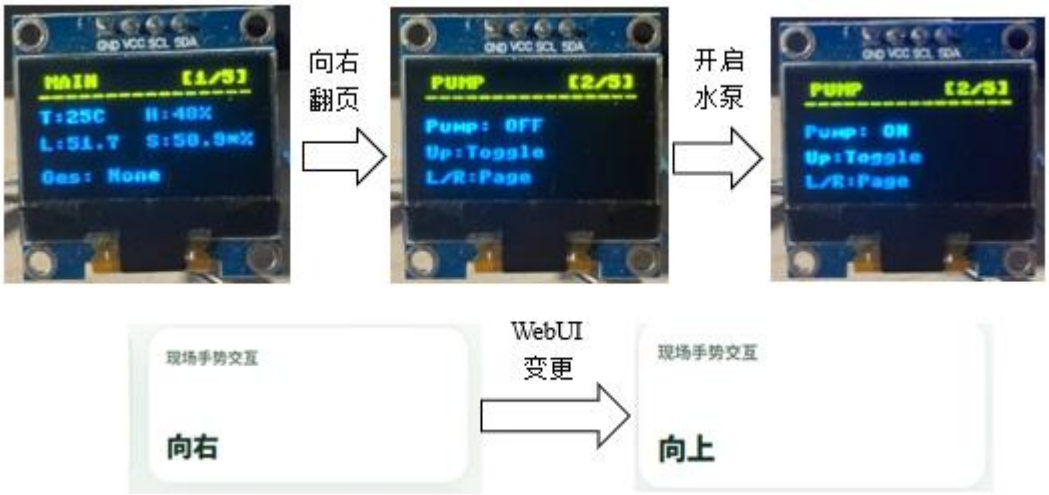

图 6.5　手势识别测试结果

另外，还对 LED 灯带的自动环境补光功能做了测试。如上图所示，通过远程下发自动模式指令，人为模拟弱光环境之后，系统根据实时光照数据自动点亮灯带，并同步状态显示，很好地验证了边缘端自主决策和控制能力。

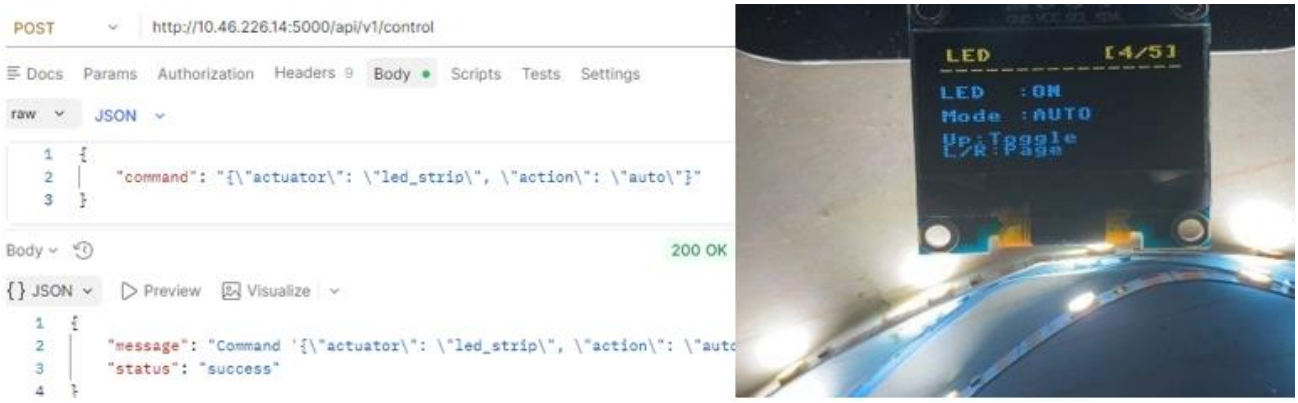

图 6.6　LED 灯带补光测试的 Postman 和实物图

6.2.3 平台服务与扩展功能测试

该测试主要验证 Web 页面访问、历史数据查询、MQTT 数据同步、图像分析、本地问答、公网 app 等功能。

表 6.3　平台服务与扩展功能测试结果表

测试项目	预期结果	实测结果
环境时序数据追溯	实时曲线绘制	正常绘制近期 200 条数据
云端公网 app 看板	正常显示传感器数据	实时同步各个传感器数据
图像分析	能输出颜色识别与阶段判断	分析出目标粉色花朵
边缘问答	能根据环境数据生成建议	Qwen 专家建议：环境适宜藏红花生长，无需立刻浇水。

对环境时序数据追溯功能进行测试，主要是检验平台对于历史数据的可视化展示能力。测试时，通过浏览器访问 Web 监控平台。如上图所示，前端页面成功获取并绘制出最近的 200 条环境数据。左侧折线图清楚地显示出温度、湿度的变化趋势，右侧折线图清楚地显示出光照、土壤水分的变化趋势。测试结果符合预期，证明该系统可以稳定地完成环境数据的可视化追溯。

图 6.7　Web 端环境时序数据追溯的折线图

其次，对云端公网 app 看板进行了测试。下图中通过手机扫描二维码访问云端公网节点，移动端页面可以实时同步并且正常显示边缘端上传的温湿度、光照、土壤水分等各项传感器最新的数据，成功实现了脱离局域网的异地远程监控。

图 6.8　云端公网 app 看板

以下对图像视觉分析的测试，结合系统抓拍的实测图可知，边缘端视觉模块可以正确地提取出花朵的颜色特征，可以准确地框选出目标粉色花朵，并且可以对不同的颜色目标进行幼苗期（绿色）、生长期（粉色）、成熟期（红色）阶段的判断，达到预期的视觉辅助诊断效果。

图 6.9　幼苗期

图 6.10　生长期

图 6.11　成熟期

最后对边缘 Qwen 模型问答进行了测试。下图是点击页面操作按钮后，本地部署的大模型能够实时根据采集到的土壤湿度等环境参数生成辅助决策建议，并输出“环境适宜藏红花生长，无需立刻浇水”的诊断结果。该结果证明系统有较好的边缘智能辅助分析能力。

图 6.12 Webapp 端 Qwen 大模型专家系统测试

6.3 测试结果分析

从系统测试的过程可以看出，本文设计的智能培育系统可以达到环境监测、自动控制、数据存储、可视化管理等主要目的。如果实测结果表明温湿度、光照和土壤湿度数据可以稳定刷新，可以持续写入数据库，那么说明设备端采集链路和边缘端数据处理链路工作正常；如果控制命令可以被设备端正确解析并执行器动作和日志记录一致，则说明系统控制闭环设计合理。

在自动灌溉测试中，如果系统可以当土壤湿度小于设定阈值的时候自动启动水泵，在达到浇水时长之后自动关闭，并且满足冷却时间的约束，那么说明边缘端策略线程可以稳定工作。如果图像分析模块可以正确输出颜色识别结果，在页面上显示阶段判断，那么系统就具备了一定的轻量视觉智能。

但是系统也存在一些不足之处。DHT11 的测量精度较低，电容式土壤湿度传感器还需要结合实际情况做更细致的标定，图像分析目前主要是根据颜色规则，对于复杂的背景还存在鲁棒性不足的问题，云端同步功能目前只是数据上报，还没有建立完整的云端业务平台。这些问题也给之后的工作指明了方向。

7 总结和展望

7.1 总结

本文根据藏红花全生长周期智能培育需求，设计并实现了一个边缘计算的智能培育系统。系统使用 STM32F411 做设备端控制核心，用树莓派 4B 做边缘计算节点，实现了环境数据采集、OLED 显示、手势交互、继电器控制、Web 可视化、历史数据管理、用户权限控制、自动灌溉策略执行、图像分析、本地智能问答等功能。

经过系统的方案设计、硬件连接实现、软件平台开发、功能测试可知，本文所建立的系统有如下特点，一是分层结构的设计比较清楚，二是实现了从环境感知到控制执行的闭环，三是边缘端具备较强的本地自治能力，即使云端连接断开也能继续工作，四是系统具有较好的扩展性，可以增加更多的传感器、执行器以及智能分析模块。本文基本达到了预期的研究目的，给小型特色作物的智能培育提供了一种可行的实现途径。

7.2 展望

本文建立的藏红花边缘计算培育系统虽然已经实现了从底层采集到云端同步的全流程，但是它在实际农业生产中还存在着一些不足。首先就是底层感知节点的精度问题，目前使用的 DHT11 模块在长时间高湿环境下容易出现数据漂移，后续可以使用工业级温湿度传感器，并对土壤 ADC 采样加入动态标定算法来消除电极极化误差。第二，目前的自动化灌溉大多使用单一的土壤水分阈值触发方式，将来会依靠 SQLite 中积累起来的时序数据以及本地运行的大模型（Qwen）来实现对光照、温湿度和生长周期的综合判断。目前视觉监控主要是用 OpenCV 提取色彩特

征来判断花期，容易受到环境光线的影响，之后会在树莓派上部署更加鲁棒的轻量化目标检测视觉模型。另外，系统目前的 MQTT 互联只实现了单向的数据汇聚，需要以后打通双向鉴权控制，针对高湿应用场景做专属 PCB 的三防设计和外壳封装，提高硬件的工程可靠性。

7.3 个人总结与反思

此课题的设计对嵌入式开发、传感器应用、Web 后端、数据库、图像处理、物联网通信等方面的知识进行了综合应用，也更加清楚地认识到系统集成与单点功能实现的不同。项目推进的过程中，由于调试硬件，兼容接口，部署环境以及多模块联调所耗费的时间大大超过计划，说明工程实践不只是对编程能力的考核，也是对问题定位，方案取舍，整体规划能力的检验。

另外，也发现自身存在很多不足，即对于硬件原理的分析、传感器精度的处理、系统抗干扰的设计以及实验数据的量化分析等方面存在较大的欠缺。此后无论是在继续深造还是在工程开发过程中，都需要加强对于底层原理的理解以及系统的实验能力。此设计使我完成了比较完整的智能物联网项目，也对我未来从事软硬件结合方面学习与实践有更清晰的认识。

参考文献

[1] 汤朝婧,唐燕.基于物联网的智能花卉栽培系统设计研究[J].现代农业装备,2024,45(01):71-75.

[2] 周德锋.基于物联网的智慧农业温室大棚环境智能调节管理系统设计[J]. 办公自动化,2023,28(10):58-60.

[3] 顾廷炜,汤明宏,孙晓冬.智能传感器技术应用现状与发展趋势综述[J].物联网技术,2025,15(01):59-63.DOI:10.16667/j.issn.2095-1302.2025.01.014.

[4] 黄成龙,柯宇曦,华向东,等.边缘计算在智慧农业中的应用现状与展望[J].农业工程学报,2022,38(16):224-234.

[5] 吴小凤,王天淼,朱文秀.基于 Flask 框架的监控平台可视化设计研究[J].工业控制计算机,2024,37(03):90-91.

[6] 景子奇,邹兆年.嵌入式数据库 SQLite 上多版本并发控制的设计与实现[J].计算机应用,2022,42(01):140-147.

[7] 郭翠娟,暴宁,荣锋.基于 MQTT 的物联网平台研究与设计[J].计算机工程与设计,2022,43(08):2378-2384.DOI:10.16208/j.issn1000-7024.2022.08.036.

[8] 张代伟.基于 OpenCV 的视觉检测系统应用[J].机电信息,2024,(01):7-10.DOI:10.19514/j.cnki.cn32-1628/tm.2024.01.002.

[9] 任海玉,刘建平,王健,等.基于大语言模型的智能问答系统研究综述[J].计算机工程与应用,2025,61(07):1-24.

[10] 徐帅驰,郑恭明.MicroPython 与云平台在物联网应用领域的开发与应用[J].单片机与嵌入式系统应用,2021,21(09):33-36.

[11] 冼进,冼允廷.基于 STM32 的智慧农业大棚系统设计[J].现代电子技术,2023,46(04):70-74.DOI:10.16652/j.issn.1004-373x.2023.04.013.

[12] 付文新,王洪丰.基于 STM32 单片机和 DHT11 温湿度传感器的温湿度采集系统的设计与实现[J].光源与照明,2022,(03):119-121.

[13] 贾书香.基于 STM32 的手势识别控制系统的设计[J].黑龙江科学,2023,14(02):78-81.

[14] 杜继明.Python 语言的 Web 开发应用分析[J].数字通信世界,2022,(01):76-78.

[15] 杨一帆.服务于 MySQL 数据库的在线监测系统设计[J].自动化技术与应用,2022,41(10):179-182.DOI:10.20033/j.1003-7241.(2022)10-0179-04.

[16] Chung H ,Kim H J ,Ahn J , et al.Resource-Constrained Edge AI Solution for Real-Time Pest and Disease Detection in Chili Pepper Fields[J].Agriculture,2026,16(2):223-223.DOI:10.3390/AGRICULTURE16020223.

[17] Li W ,Ai X ,Kang G , et al.LLM-Based RESTful Web API Service Discovery[J].Service Oriented Computing and Applications,2025,(prepublish):1-15.DOI:10.1007/S11761-025-00485-4.

[18] Li L ,Liu Z C ,Cao K , et al.Artificial intelligence of things for smart agriculture with optimal embedded edge intelligence[J].Computers and Electrical Engineering,2026,133111039-111039.DOI:10.1016/J.COMPELECENG.2026.111039.

[19] Liu D ,Zhou X ,Chen D , et al.Smart indoor occupancy detection based on optimized camera placement, multi-view de-duplication, and large language model semantic understanding[J].Engineering Applications of Artificial Intelligence,2026,169114157-114157.DOI:10.1016/J

[20] Li Y ,Wu S .Design of an Intelligent Rice Seedling Greenhouse Environmental Control System: A Solution Based on STM32[J].Academic Journal of Engineering and Technology Science,2025,8(5):DOI:10.25236/AJETS.2025.080504.

致谢

首先，要对姚老师指导工作表示最真诚的感谢，在选题、方案设计、论文写作、毕业设计等各个环节上都得到了老师很大的支持和鼓励。姚老师对整个研究思路、技术路线、论文结构等提出了许多有见地的指导，使本次毕业设计可以比较顺利地完成。同时向嘉立创(JLC)、Raspberry Pi Foundation 等提供实验环境、实验教学平台的同学及在硬件调试、系统检测、资料查阅等各个方面的交流。还要感谢父母对我的关心和照顾，才能使我不顾一切地进行毕业论文的研究。最后我要向所有对本次毕业设计作出过指导和支持的老师表示感谢，并向帮助我完成毕业设计的同学们表示敬意。正是不断调试、修改、完善，才使得智能硬件、边缘计算、物联网系统的开发有了更加深刻的认识，也将是我以后学习、发展的基础。